OS Orange Assignment

Name: Apoorva Kogta
SRN: PES2UG24CS078

https://github.com/2306apk/PES2UG24CS078-pes-vcs

PHASE 1

1a

```
user@ubuntu-vm:~/pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b

```
user@ubuntu-vm:~/pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

PHASE 2

2a

```
user@ubuntu-vm:~/pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2b

```
user@ubuntu-vm:~/pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
user@ubuntu-vm:~/pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650ed
a471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
user@ubuntu-vm:~/pes-vcs$
```

PHASE 3

3a

```
user@ubuntu-vm:~/pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  tree.c
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  object.c
  untracked:  test_tree
  untracked:  Makefile
  untracked:  pes.h
  untracked:  pes.c
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  commit.c
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  index.h
  untracked:  test_objects.c
```

3b

```
user@ubuntu-vm:~/pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776485761 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776485761 6 file2.txt
user@ubuntu-vm:~/pes-vcs$
```

PHASE 4

4a

```
./pes log
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.
o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Initialized empty PES repository in .pes/
Committed: 32d3e0093441... Initial commit
Committed: 0d5a7be95845... Update file1
commit 0d5a7be9584560d7e04e0aaa04a2c4f31119b1cd04876ea2828a18e0a026bb52
Author: Apoorva <PES2UG24CS078>
Date:   1776486606

    Update file1

commit 32d3e0093441eded1c1b32986f882a8a67151944c24a3e2c10e4be9ef3cdf94a
Author: Apoorva <PES2UG24CS078>
Date:   1776486606

    Initial commit
```

4b

```
user@ubuntu-vm:~/pes-vcs$ find .pes -type f
.pes/objects/0d/5a7be9584560d7e04e0aaa04a2c4f31119b1cd04876ea2828a18e0a026bb52
.pes/objects/8d/cdf2e4a1675e50aa7c30cef00990b702b955a90f250a2f7b1beeba8692ed0f
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/32/d3e0093441eded1c1b32986f882a8a67151944c24a3e2c10e4be9ef3cdf94a
.pes/objects/83/65c42d37b4ea9e9ae82c621ee1779945591e7e988c5c9ec39f711118b32df9
.pes/index
.pes/refs/heads/main
.pes/HEAD
```

4c

```
user@ubuntu-vm:~/pes-vcs$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
0d5a7be9584560d7e04e0aaa04a2c4f31119b1cd04876ea2828a18e0a026bb52
user@ubuntu-vm:~/pes-vcs$
```

PHASE 5

5.1

First, check if the branch file exists in `.pes/refs/heads/`. Read the commit hash from it and update `.pes/HEAD` using a temporary file and rename for safety.

Then, read the commit object, get its root tree, and recursively expand all tree objects to map file paths to blobs.

Next, update the working directory:

- Write blob contents into files with correct permissions
- Remove files that are no longer tracked
- Create directories if needed

Finally, rebuild the index to match the target tree, including metadata.

This is complex because it must handle deletions, nested directories, permissions, and avoid overwriting uncommitted changes.

5.2

For each tracked file, perform a 3-way comparison:

- Base → current HEAD version
- Staged → version in index
- Working → current file

Compare current commit with target commit.

Before checkout:

- If a file has local changes and target also modifies it → stop
- If an untracked file will be overwritten → stop

This prevents loss of uncommitted work.

5.3

In detached HEAD, `.pes/HEAD` stores a commit hash instead of pointing to a branch.

New commits can still be created, but no branch tracks them.

These commits can become unreachable. To recover, create a branch pointing to them or switch to a branch after saving.

Otherwise, garbage collection may delete them.

PHASE 6

6.1

Use mark-and-sweep:

- Mark phase: Start from branch tips and traverse commits, trees, and blobs, marking all reachable objects
- Sweep phase: Delete all unmarked objects from `.pes/objects`

A hash set is used to track visited objects efficiently

6.2

A race condition can occur during commit:

- Objects are written first
- Then branch reference is updated

If garbage collection runs before the reference update, it may delete new objects thinking they are unused.

To prevent this:

- Use locks and synchronization
- Maintain reflogs or grace periods
- Delay deletion of recent objects

This ensures objects are safe before references are updated.